

Project 10 (Bonus) — MemoryKeeper-Persistent-Assistant-That-Remembers-You-Across-Sessions

Title: MemoryKeeper Persistent Assistant That Remembers You Across Sessions

Type: Single-agent chat assistant (LangGraph graph, checkpointer + long-term store) · **Stack:** Python, LangGraph, `langchain-openai`, `SqliteSaver`, `InMemoryStore`, `python-dotenv`, `pydantic` · **Level:** Intermediate **Tagline:** A terminal chat assistant that remembers real facts about you across separate runs of the program — not just within one conversation.

Overview

MemoryKeeper is a terminal chat assistant that remembers real facts about a specific user across completely separate runs of the program — not just within a single conversation. It combines a LangGraph checkpointer for short-term, per-conversation memory with a separate long-term Store keyed by user ID, so closing the program and reopening it tomorrow doesn't reset who the assistant thinks it's talking to. It decides for itself which facts are actually worth keeping, avoids saving the same fact twice in different words, and gives the user a real, verified way to be forgotten. This closes a real gap in most chat demos: any assistant a user expects to "remember me" the next time they open it — a support bot, a coding assistant, anything with a return visit built into the product.

Features

- Persists one conversation's state across process restarts using a `SqliteSaver`-backed LangGraph checkpointer
- Recalls facts about a specific user in a brand-new conversation thread, via a separate long-term Store keyed by `user_id`
- Judges, per message, whether something is durable enough to remember instead of saving everything said
- Overwrites outdated facts under fixed keys instead of creating duplicate, contradicting memories
- Supports a `forget_me` command that deletes every stored fact for a user, verified by a fresh process afterward
- Keeps per-conversation state and per-user long-term memory as two clearly separate systems, not one blended store

Tech Stack

- Python
- LangGraph
- `langchain-openai`
- `SqliteSaver` (file-backed checkpointer)
- `InMemoryStore` (long-term memory store)
- `python-dotenv`
- `pydantic`

Architecture

MemoryKeeper is a single LangGraph graph with two independent memory systems attached to it. A `SqliteSaver` checkpointer holds one conversation's full message history, keyed by `thread_id`, and survives the process being killed and restarted on the same thread. Next to it sits an `InMemoryStore` (swappable for `PostgresStore` in production), keyed by `user_id`, holding a small number of distilled facts. Before the model answers, a `recall_node` reads the Store and injects any known facts about the user into the system prompt. After the model answers, a `write_back_node` judges whether the just-said message contains anything durable, and if so writes it to the Store under a fixed key so a reworded restatement overwrites rather than duplicates it. A `forget_me` command deletes every key in a user's Store namespace directly, without going through the model.

```
START -> recall_node -> chat_node -> write_back_node -> END
      |                   |
      |                   |
reads Store (by user_id)  writes Store (by user_id)
checkpointer holds the full conversation, keyed by thread_id
```

Charter (what this project is)

Build a terminal chat assistant — reusing Doc01's config/logging, Doc04's chat patterns, and Doc09's LangGraph graph structure — that remembers real facts about the user **across separate runs of the program**, not just within one conversation. Close the program. Run it again tomorrow, as a brand-new conversation. It should still remember your name, and what you told it last time. Doc07's "Agent memory" topic named this problem and set it aside on purpose, for later. Doc09's "Long-term memory" topic just introduced the real mechanism for solving it: `langgraph.store`, a `Store` kept separate from the per-conversation checkpointer, keyed by user instead of by thread. This project builds that mechanism all the way through — read facts in, decide what's actually worth saving, write facts out, and give the user a real way to be forgotten.

Jump to: [Setup](#) · [Step 1](#) · [Step 2](#) · [Step 3](#) · [Step 4](#)

The Story — what you're actually building

Picture a chat assistant with a good short-term memory. Inside one conversation, it remembers everything you said five minutes ago — that's a checkpointer, and Doc09 already taught you how to build one. Now picture closing that conversation, coming back tomorrow, and starting a brand-new one. A checkpointer alone gives you a stranger. It has no idea who you are, even though you told it your name yesterday.

That's the actual gap this project closes. MemoryKeeper is a small terminal chat assistant with two kinds of memory working together, not one. The checkpointer still handles one conversation's back-and-forth, exactly like Doc09 taught. But sitting next to it is a second thing: a long-term **Store**, keyed by user instead of by conversation, that survives no matter how many separate runs you start. Before MemoryKeeper answers you, it checks the Store for anything it already knows about you. After you say something worth remembering, it writes a short, distilled fact back to the Store — not the whole conversation, just the part worth keeping. And because a real memory system has to let people leave, MemoryKeeper also has a `forget_me` command that actually deletes what it knows about you, for good.

The Steps below build this up in order. Step 1 proves the limitation first: a normal LangGraph chat graph, with only a checkpointer, that forgets everything the moment you start a new thread. Step 2 adds the Store and a node that reads it, so a brand-new thread can still "know" you. Step 3 adds the harder half: deciding, on its own, what's actually worth writing back, and not saving the same fact twice in slightly different words. Step 4 adds forgetting — a real delete, tested by proving a fresh run afterward truly knows nothing.

Before you read further — think about it yourself: if a brand-new conversation (a new `thread_id`) can still know your name, where does the checkpointer's job end and the Store's job begin — are they really the same "memory," or two separate systems doing two separate jobs? And if the assistant has to decide, on its own, whether something you just said is worth remembering forever, what should make it say no — should it save "I'm in a bad mood today" the same way it saves your name?

What you're actually building, in one line: a terminal chat assistant that keeps two separate memories — one for the current conversation, one for facts about you that survive across separate runs.

Why this needs to exist: a model's context window is finite and every extra token costs money on every call, so you can't just keep feeding it the whole history of every past conversation forever. You need a small, separate place to keep only the facts worth keeping.

When you'd reach for this at a real job: any assistant a user expects to "remember me" the next time they open it — a support bot that recalls a customer's plan, a coding assistant that recalls a user's preferred setup, anything with a return visit built into the product.

How it works, mechanically: a checkpointer saves one conversation's full message history, keyed by thread ID; a separate Store saves a small number of distilled facts, keyed by user ID, that one node reads before answering and another node writes to after.

Why not just do it some simpler/different way: you could just keep stuffing the whole conversation history into context and let the model "remember" that way — but context windows are limited in size, and cost grows with every token you send on every call, so this stops working the moment there's more history than fits. You could also assume the checkpointer alone is enough — but its state lives inside one thread, and the moment a new thread starts (a new day, a new session), that state is gone, no matter how much you saved. Only a store keyed by user, not by thread, survives that.

Where This Fits

This is a bonus/portfolio project, not part of the main 5-project arc (Projects 1-5), and it isn't scored the way Project 6 tests architecture on a new domain. It's built directly on [09_langgraph](#)'s "Long-term memory: remembering across separate conversations" topic — the real mechanism (`langgraph.store`) for a problem [07_ai_agents](#)'s "Agent memory: what actually persists between calls" topic named early on and explicitly deferred, because at that point in the curriculum there was no graph to hang it on yet. Now there is. If you haven't read Doc09's Core Concepts yet, read them first — this file assumes you already know what a checkpointer is, and builds the second, separate memory system next to it.

What's new here is turning "the Store exists" into a full, working loop: read from it before answering, decide what's worth writing after answering, and give the user a real way to erase it. This belongs in your portfolio as proof you can build a memory system that does more than store text — one that also makes a judgment call about what's worth keeping, and respects a user who wants to be forgotten.

Setup (do this once, before Step 1)

```
cd project_10_memorykeeper
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install langgraph langgraph-checkpoint-sqlite langchain-openai python-dotenv
pip freeze > requirements.txt
```

Same `.env` setup as every other project:

```
OPENAI_API_KEY=sk-...
LOG_LEVEL=INFO
```

And the same `.env.example` with the key names but no real secret.

`langgraph-checkpoint-sqlite` is a separate package from `langgraph` itself — it's what gives you `SqliteSaver`, a checkpointer backed by a real file on disk instead of a plain Python dict (`MemorySaver`, Doc09's default) that disappears the moment the process ends. You need the file-backed one here: Step 1's whole point is proving something survives being a completely new `python main.py` process.

A Real Example (so this isn't just theory)

Use this scenario, or one close to it, for every step below:

Scenario: You are the user. Your `user_id` is a fixed string, like `"sam"`, that you pass to MemoryKeeper every time you run it (an argument, or an env var — your choice). Across several separate runs:

1. **Run 1, thread A:** "Hi, I'm Sam. I prefer being called Sam, not Samuel." Chat a little more, then close the program.
2. **Run 2, thread A again (same thread ID):** Ask "what's my name?" — this should work even with only a checkpointer, since it's the *same* conversation continuing (Step 1's baseline).
3. **Run 3, thread B (a brand-new thread ID, same `user_id`):** Ask "what's my name?" again — with only a checkpointer (Step 1), this fails: brand-new thread, brand-new empty state. With the Store wired in (Step 2 onward), this should work: MemoryKeeper still knows you're Sam, in a conversation it has never seen before.
4. **Run 4, thread C:** Say something small and forgettable, like "I think I'll have pasta tonight." Confirm (Step 3) that this does **not** get written to long-term memory — it's not a durable fact, it's just today's dinner.
5. **Run 5, thread D:** Say "actually, call me Sammy instead." Confirm (Step 3) this updates the existing name fact instead of sitting alongside it as a second, contradictory one.
6. **Run 6:** Type `forget_me`. Confirm MemoryKeeper deletes everything it knows about `"sam"`.
7. **Run 7, a brand-new thread:** ask "what's my name?" one more time — it should now have no idea (Step 4's test).

Why This Project Is Good for Your Portfolio

- **What problem it solves:** most chat demos quietly reset the moment you close the terminal. This one doesn't — it remembers real facts about a specific user across completely separate runs, and can be told to forget them again.
- **Why it matters:** a checkpointer alone is not "memory" in the way most users mean it — it's one conversation's scratchpad. Knowing the difference between per-conversation state and per-user

long-term memory, and building both correctly, is exactly what separates a toy chatbot from something you'd trust with real user data.

- **When you'd build something like this at a real job:** any assistant a user talks to more than once — a support bot that remembers a customer's plan and past issues, an internal tool that remembers a colleague's preferences, anything with a "remember me" expectation and a legal or ethical obligation to actually delete data on request.
- **How it's built:** a LangGraph chat graph with a `SqliteSaver` checkpoint for one conversation's state, an `InMemoryStore` (or `PostgresStore` in production) namespaced by `user_id` for facts that outlive any one thread, a node that decides what's worth saving, and a `forget_me` path that deletes a user's Store entries for real.

Problems you'll likely run into, and how to fix them:

Problem	Fix
A new thread ID still "remembers" the last conversation	Check you're actually starting a new thread — a repeated <code>thread_id</code> in your test script is the checkpoint doing its job correctly, not the Store. Change <code>thread_id</code> between runs to genuinely test the Store, and keep <code>user_id</code> fixed
The Store node saves something, but a later run can't find it	Confirm the namespace you write to and the namespace you read from are built the exact same way — (<code>user_id</code> , "memories") has to match on both sides, including <code>user_id</code> being the same type (a stray <code>int</code> vs <code>str</code> mismatch is a common cause)
Every single message gets written to long-term memory, including "ok" and "thanks"	The write-back node needs an actual judgment step, not "always save the last message" — ask a small, separate LLM call something like "is there a durable fact here worth remembering forever? If not, say so" before writing anything
The same fact gets saved twice, worded slightly differently ("Sam" vs "call me Sam")	Save facts under a small number of fixed, meaningful keys (like "preferred_name"), not a new random key per save — writing to the same key overwrites the old value instead of piling up near-duplicates
<code>forget_me</code> "succeeds" but a later run still knows the user's name	Deleting one key isn't enough if you saved multiple facts under multiple keys — list every key in the user's namespace and delete each one, then prove it by actually running a fresh process afterward, not just checking the delete call didn't raise an error

Built During These Documents

[01_python_foundations](#) → [04_openai_api](#) → [09_langgraph](#)

Plan Before You Code

Same process as every project (see [15_five_projects_index](#)): Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks — before writing any graph code. Before you open your editor, sketch on paper the two memory systems side by side: what lives in the checkpointer's state (this conversation's messages), and what lives in the Store (facts about this user, independent of any one conversation). If you can't say, for a given piece of information, which of the two it belongs in, that's worth settling before Step 1.

How To Build This — Step by Step

Step 1 — A Normal Graph With Only Short-Term (Checkpointer) Memory

Project: MemoryKeeper-Persistent-Assistant-That-Remembers-You-Across-Sessions — Step 1 of 4: A Normal Graph With Only Short-Term (Checkpointer) Memory

What this step does: builds a plain, working LangGraph chat graph with a `SqliteSaver` checkpointer, and proves — on purpose — both what it can and can't do: it remembers everything inside one thread, and remembers nothing at all in a brand-new one. **Why this step matters:** you can't appreciate what long-term memory adds until you've felt its absence for yourself. This step is the baseline the rest of the project measures itself against — every later step gets compared back to "what Step 1 could and couldn't do." **When you'll hit this for real:** the first time a user says "wait, didn't I already tell you this yesterday?" — and you have to explain, honestly, that yesterday's conversation and today's are two completely different threads as far as the checkpointer is concerned. **Read first:** [09_langgraph Core Concepts — "Checkpoints: saved state that survives a pause"](#), [09_langgraph Core Concepts — "State: one typed object flowing through the graph"](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write `state.py`: a `ChatState(TypedDict)` with `messages: list` (use `Annotated[list, add_messages]`, the same reducer pattern Doc09 uses, so new messages append instead of overwriting). 2. Write `graph.py`: a `build_graph(checkpointer)` function with one node, `chat_node`, that calls `ChatOpenAI(model="gpt-4o-mini")` on `state["messages"]` and returns the reply appended to `messages`. Wire `START → chat_node → END`. Compile with `builder.compile(checkpointer=checkpointer)`. 3. In `main.py`, build a real `SqliteSaver`: `conn = sqlite3.connect("memory_keeper.db", check_same_thread=False)`, then `SqliteSaver(conn)`. Pass it into `build_graph`. 4. Write a small terminal loop: read a line of input, call `graph.invoke({"messages": [...]}, config={"configurable": {"thread_id": THREAD_ID}})`, print the reply. Hardcode a `THREAD_ID` for now (Step 2 makes this a real per-run ID). 5. Prove both halves of the baseline: run `main.py`, chat a little, close it, run it again with the **same** `THREAD_ID` — confirm it remembers. Then run it once more with a **different** `THREAD_ID` — confirm it has no memory of the earlier conversation at all. Write both checks into `test_memory_keeper.py` so you don't have to eyeball it by hand every time.

Your files after Step 1:


```

project_10_memorykeeper_persistent_memory/
├── .env / .env.example
├── config.py                (reused pattern from Doc01)
├── logging_setup.py        (reused pattern from Doc01)
├── state.py                → ChatState(TypedDict): messages (with add_message
├── graph.py                → build_graph(checkpointer) -> compiled graph, one
├── main.py                 → terminal chat loop, fixed thread_id, SqliteSaver
└── test_memory_keeper.py   → confirms same thread_id remembers, different thr

```

Step 2 — Add a Long-Term Store, and a Node That Reads From It

Project: MemoryKeeper-Persistent-Assistant-That-Remembers-You-Across-Sessions — Step 2 of 4:
Add a Long-Term Store, and a Node That Reads From It

What this step does: wires in an `InMemoryStore`, namespaceed by `user_id`, and adds a node at the very start of the graph that reads any saved facts about the current user and injects them into the system prompt — before the model ever answers. **Why this step matters:** this is the actual fix for Step 1's gap. A checkpointer can't help a brand-new thread, because it's keyed by thread on purpose. The Store is keyed by `user_id` instead, so it doesn't care how many separate threads that user has ever started. **What's new vs. Step 1:** a `store.py` module, a `recall_node` that runs before `chat_node`, and a real `user_id` (instead of only a `thread_id`) flowing through `main.py`. **What stays the same:** `chat_node` itself, and the checkpointer from Step 1 — you're adding a second memory system next to the first one, not replacing it. **When you'll hit this for real:** any assistant that talks to the same person more than once, across more than one session — a returning customer, a returning colleague, anyone the product genuinely expects to come back. **Read first:** [09_langgraph Core Concepts — "Long-term memory: remembering across separate conversations"](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write `store.py`: `get_store()` returning a module-level `InMemoryStore()` (write a comment noting `PostgresStore` is the production swap, same shape, different constructor), plus `namespace_for(user_id)` returning `(user_id, "memories")` — one function, used everywhere a namespace is needed, so a typo in one place can't silently break a lookup somewhere else. 2. Update `state.py`: add `user_id: str` to `ChatState`. 3. Write `recall_node(state, store)`: calls `store.search(namespace_for(state["user_id"]), query=...)` (or `store.get(...)` for known keys, like `"preferred_name"`), builds a short system message like `"Known facts about this user: ..."` from whatever it finds, and puts it first in `messages` if it isn't already there. If nothing is found, add nothing — don't inject an empty or awkward "I know nothing about you" line. 4. Update `graph.py`: `START` → `recall_node` → `chat_node` → `END`. Since `recall_node` needs the store and `chat_node` doesn't need to change, use a closure or `functools.partial` to bind the store into `recall_node` when building the graph, the same way you bound the checkpointer in Step 1. 5. Seed a fact by hand for testing (call `store.put(namespace_for("sam"), "preferred_name", {"text": "Sam"})` once, outside the graph), then update `main.py` to

accept a `user_id` and generate a **new** `thread_id` every run. Run it, ask "what's my name?" in a thread that has never existed before, and confirm the answer is correct — proving recall, not the checkpointer, is what made it work.

Your files after Step 2:

```
project_10_memorykeeper_persistent_memory/
├─ config.py
├─ logging_setup.py
├─ state.py           → adds user_id: str
├─ store.py           → get_store(), namespace_for(user_id)
├─ graph.py           → adds recall_node before chat_node
├─ main.py            → new thread_id every run, fixed user_id, seeds a
└─ test_memory_keeper.py → adds: a brand-new thread_id still recalls a seed
```

Step 3 — Deciding What's Worth Remembering, and Writing It Back

Project: MemoryKeeper-Persistent-Assistant-That-Remembers-You-Across-Sessions — Step 3 of 4: Deciding What's Worth Remembering, and Writing It Back

What this step does: adds a node that looks at what was just said and decides, on its own, whether anything genuinely durable came up — a name, a stated preference — and if so, writes just that distilled fact to the Store. Most messages write nothing at all. **Why this step matters:** Step 2 proved reading from the Store works, but everything in it so far was seeded by hand. A real system has to decide, itself, what's worth keeping — and the real design problem is saying no far more often than yes. **What's new vs. Step 2:** a `memory_writer.py` module and a `write_back_node`, running after `chat_node`. **What stays the same:** `recall_node`, and the checkpointer — this step only adds a way for facts to get into the Store on their own, it doesn't touch how they get read back out.

When you'll hit this for real: the moment "remember what I tell you" stops being a demo script with a hand-seeded fact, and has to work on whatever a real user actually types, most of which isn't worth remembering at all. **Read first:** [09_langgraph Core Concepts — "Long-term memory: remembering across separate conversations"](#) (the paragraph on deciding what's worth saving), [07_ai_agents Core Concepts — "Agent memory: what actually persists between calls"](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write `memory_writer.py`: `judge_message(text: str) -> dict | None`, a small, separate `ChatOpenAI` call (structured output — a pydantic model with `worth_remembering: bool`, `key: str | None`, `value: str | None`) that looks at one message and decides if it names a durable fact (a name, a stated preference, a correction to a fact already known) versus something that only matters today (a mood, a one-off request, small talk). Write the prompt with real examples of both — "I'm Sam" is worth remembering, "I think I'll have pasta tonight" is not — models guess better with 2-3 concrete examples than with an abstract rule alone. 2. Decide your fact keys up front, and keep the list short and fixed (`"preferred_name"`, `"stated_preference"`, and so

on) — this is the actual answer to "how do you avoid saving the same fact twice in slightly different words": you don't generate a new key per save, you write to one of a small number of known keys, so a later, re-worded version of the same fact overwrites the old one at `store.put(namespace, key, value)` instead of sitting next to it as a second, half-contradicting entry. 3. Write `write_back_node(state, store)`: after `chat_node` runs, call `judge_message` on the user's latest message, and if `worth_remembering` is true, `store.put(namespace_for(state["user_id"]), key, {"text": value})`. If false, do nothing — log at `DEBUG`, not `INFO`, so a normal run's logs aren't full of "decided not to save" noise. 4. Update `graph.py`: `START` → `recall_node` → `chat_node` → `write_back_node` → `END`. 5. Test both directions on purpose: a message that should get saved (confirm a later, brand-new thread recalls it correctly), and one that shouldn't (confirm the Store's contents are unchanged after it). Also test the overwrite case: state your name, then correct it in a later message — confirm the Store ends up with one current value, not two.

Your files after Step 3:

```
project_10_memorykeeper_persistent_memory/
├─ config.py
├─ logging_setup.py
├─ state.py
├─ store.py
├─ memory_writer.py          → judge_message(text) -> dict | None, structured-
├─ graph.py                  → adds write_back_node after chat_node
├─ main.py
└─ test_memory_keeper.py     → adds: a durable fact gets saved, a forgettable
```

Step 4 — Forgetting, and a Privacy-Respecting Delete

Project: MemoryKeeper-Persistent-Assistant-That-Remembers-You-Across-Sessions — Step 4 of 4: Forgetting, and a Privacy-Respecting Delete

What this step does: adds a real `forget_me` command that deletes everything the Store knows about the current user, and proves it worked the honest way — by running a brand-new process afterward and confirming it truly has no memory left, not just that the delete call didn't raise an error.

Why this step matters: a memory system that can only add and never remove isn't safe to put in front of a real user. People correct facts, and people ask to be forgotten — both need an actual, working delete, not a feature that exists in name only. **When you'll hit this for real:** a user asks "please forget what I told you" or a fact you saved turns out to be wrong, and the honest answer to "can we actually delete that?" has to be yes, not "well, technically it's still in there somewhere." **Read first:** [09_langgraph Core Concepts — "Long-term memory: remembering across separate conversations"](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Add `forget_user(user_id)` to `store.py` : call `store.search(namespace_for(user_id))` to list every item actually stored for that user, then `store.delete(namespace_for(user_id), item.key)` for each one. Don't assume there's only ever one key — Step 3 may have written several (`preferred_name` , `stated_preference` , ...), and a delete that only clears one of them isn't a real forget. 2. In `main.py` 's terminal loop, recognize a special input, `forget_me` , that calls `forget_user(user_id)` directly (no LLM call needed — this is a plain, deterministic command, not something you want the model deciding whether to honor) and prints a clear confirmation, like `"All saved memories for sam have been deleted."` 3. Think through, in a short comment above `forget_user` , why this has to be a real delete and not a soft flag: if `forget_me` only set an `is_forgotten` marker but left the actual fact rows in place, a bug (or a change of mind by someone other than the user) could bring them right back — a real delete removes that risk entirely. 4. Update `test_memory_keeper.py` with the honest version of this test: seed a fact, confirm a fresh run recalls it, call `forget_user` , then run the recall check **again as a genuinely separate step** — confirm the Store returns nothing for that user, and a brand-new thread's `recall_node` finds nothing to inject. 5. Run the full scenario from **A Real Example** start to finish, all 7 runs, and confirm every step behaves the way it's described there.

Your files after Step 4 (final):

```
project_10_memorykeeper_persistent_memory/
├─ config.py
├─ logging_setup.py
├─ state.py
├─ store.py           → adds forget_user(user_id) -> deletes every key
├─ memory_writer.py
├─ graph.py
├─ main.py           → recognizes a forget_me command, calls forget_us
└─ test_memory_keeper.py → adds: forget_user actually empties the Store, p
```

Final Deliverable: MemoryKeeper-Persistent-Assistant-That-Remembers-You-Across-Sessions — a terminal chat assistant with two working memory systems: a `SqliteSaver` checkpointer for one conversation's state, and an `InMemoryStore` for facts that survive across completely separate runs. It decides for itself what's worth remembering, avoids saving the same fact twice, and gives the user a real, tested way to be forgotten.

Checklist Before You Call This Done

- [] A `SqliteSaver` -backed checkpointer proven to survive being a completely new `python main.py` process, on the same `thread_id`
- [] The same test, on a brand-new `thread_id` , proven to have **no** memory with only the checkpointer (Step 1's baseline)

- [] `recall_node` reads from the Store by `user_id` , not by `thread_id` , and a brand-new thread can still recall a fact saved in an earlier one
- [] `write_back_node` decides, per message, whether something is durable — a test message that shouldn't be saved is proven to leave the Store unchanged
- [] Facts are saved under a small, fixed set of keys, so a re-worded version of the same fact overwrites the old value instead of creating a duplicate
- [] `forget_me` deletes every key in a user's namespace, proven by a fresh process afterward finding nothing — not just a delete call that didn't error
- [] You can explain, for any given piece of information in this project, whether it belongs in the checkpoint's state or the Store, and why

Status

Not started. Track your progress in [../PROGRESS.md](#).

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**

Part of a 13-project multi-agent AI engineering curriculum. See the [full curriculum](#) for the complete learning path and all other projects.